

Inferring Datatypes

Ningning Xie
HIW 2019.08.23

data Maybe a = Nothing | Just a

data List a = Nil | Cons a (List a)

data Either a b = Left a | Right b

Which datatype declarations should be accepted?

What kinds do accepted datatypes have?



Haskell98

```
data AppInt f = Mk (f Int)
```

```
AppInt :: ?
```

Haskell98

```
data AppInt f = Mk (f Int)
```

```
AppInt :: ?
```

```
AppInt :: (* → *) → *
```

data Q1 a = MkQ1

data Q2 = MkQ2 (Q1 Maybe)

```
data Q1 a = MkQ1
```

```
-- Q1 :: * -> *
```

```
data Q2 = MkQ2 (Q1 Maybe)
```

```
data Q1 a = MkQ1
```

```
-- Q1 :: * -> *
```

```
data Q2 = MkQ2 (Q1 Maybe)
```

```
-- Rejected!
```

data Q1 a = MkQ1 Q2

data Q2 = MkQ2 (Q1 Maybe)

```
data Q1 a = MkQ1 Q2
```

```
-- Q1 :: (* -> *) -> *
```

```
data Q2 = MkQ2 (Q1 Maybe)
```

```
data Q1 a = MkQ1 Q2
```

```
-- Q1 :: (* -> *) -> *
```

```
data Q2 = MkQ2 (Q1 Maybe)
```

```
-- Accepted!
```

Modern Haskell

Extension	Description	
AllowAmbiguousTypes	Allow the user to write ambiguous types and the type inference engine to infer them.	
Applicative DeriveFunctor	Enable deriving for the Functor class. Implied by DeriveTraversable .	
Array Derive	ExtendedDefaultRules Use GHCi's extended default rules in a normal module.	
Bang Derive	FlexibleContext LiberalTypeSynonyms	Enable liberalised type synonyms.
Binary Derive	FlexibleContext MagicHash	OverloadedLabels Enable overloaded labels.
Block Derive	ForeignFunctionInterface MonadComprehensions	OverloadedLists Enable overloaded lists.
Capabilities Derive	ForeignFunctionInterface MonadComprehensions	OverloadedStrings Enable overloaded string literals.
Concurrency Derive	ForeignFunctionInterface MonadComprehensions	PackageImports StrictData Enable default strict datatype fields.
Concurrency Derive	ForeignFunctionInterface MonadComprehensions	ParallelListComp TemplateHaskell Enable Template Haskell.
Concurrency Derive	ForeignFunctionInterface MonadComprehensions	PartialTypeSignatures TemplateHaskellQuotes Enable quotation subset of Template Haskell .
Concurrency Derive	ForeignFunctionInterface MonadComprehensions	PatternGuards TraditionalRecordSyntax Disable support for traditional record syntax (as supported by Haskell 98) <code>c {f = x}</code>
Concurrency Derive	ForeignFunctionInterface MonadComprehensions	PatternSynonyms TransformListComp Enable generalised list comprehensions.
Concurrency Derive	ForeignFunctionInterface MonadComprehensions	PolyKinds Trustworthy Enable the Safe Haskell Trustworthy mode.
Concurrency Derive	ForeignFunctionInterface MonadComprehensions	PostfixOperators TupleSections Enable tuple sections.
Concurrency Derive	ForeignFunctionInterface MonadComprehensions	QuantifiedTypes TypeApplications Enable type application syntax.
Concurrency Derive	ForeignFunctionInterface MonadComprehensions	QuasiQuotes TypeFamilies Enable type families. Implies ExplicitNamespaces , KindSignatures , and MonoLocalBinds .
Concurrency Derive	ForeignFunctionInterface MonadComprehensions	Rank2Types TypeFamilyDependencies Enable injective type families. Implies TypeFamilies .
Concurrency Derive	ForeignFunctionInterface MonadComprehensions	RankNTypes TypeInType Deprecated. Enable kind polymorphism and datatype promotion.
Concurrency Derive	ForeignFunctionInterface MonadComprehensions	RebindableNamespaces TypeOperators Enable type operators. Implies ExplicitNamespaces .
Concurrency Derive	ForeignFunctionInterface MonadComprehensions	RecordWildcards TypeSynonymInstances Enable type synonyms in instance heads. Implied by FlexibleInstances .
Concurrency Derive	ForeignFunctionInterface MonadComprehensions	RecursiveDo UnboxedSums Enable unboxed sums.
Concurrency Derive	ForeignFunctionInterface MonadComprehensions	RoleAnnotations UnboxedTuples Enable the use of unboxed tuple syntax.
Concurrency Derive	ForeignFunctionInterface MonadComprehensions	Safe UndecidableInstances Enable undecidable instances.
Concurrency Derive	ForeignFunctionInterface MonadComprehensions	ScopedTypeVariables UndecidableSuperClasses Allow all superclass constraints, including those that may result in non-termination of the typechecker.
Concurrency Derive	ForeignFunctionInterface MonadComprehensions	StandaloneDeriving UnicodeSyntax Enable unicode syntax.
Concurrency Derive	ForeignFunctionInterface MonadComprehensions	StarIsType Unsafe Enable Safe Haskell Unsafe mode.
Concurrency Derive	ForeignFunctionInterface MonadComprehensions	StaticPointers ViewPatterns Enable view patterns.
Concurrency Derive	ForeignFunctionInterface MonadComprehensions	Strict

using

```
{-# LANGUAGE  
  ExplicitForAll  
, PolyKinds  
, ExistentialQuantification  
, TypeInType  
, TypeApplications  
#-}
```

data X :: forall a (b::*->*). a b -> *

data Y :: forall (c :: Maybe Bool). X c -> *

```
data X :: forall a (b::*->*). a b -> *
```

```
data Y :: forall (c :: Maybe Bool). X c -> *
```

data X :: forall a (b::*->*). a b -> *

data Y :: forall (c :: **Maybe** Bool). X c -> *

a :: (* → *) → *

Kind mismatch!

```
data Q :: forall (a :: f b) (c :: k) . f c -> *
```

```
data Q :: forall (a :: f b) (c :: k) . f c -> *
```

data Q :: forall (a :: f b) (c :: k) . f c -> *

data Q :: forall (f::?) (b::?) (k::?) forall (a :: f b) (c :: k) . f c -> *

```
data Q :: forall (a :: f b) (c :: k) . f c -> *
```

```
c :: k
```

```
k :: *
```

```
data Q :: forall (a :: f b) (c :: k) . f c -> *
```

```
c :: k
```

```
k :: *
```

```
f :: k -> *
```

```
data Q :: forall (a :: f b) (c :: k) . f c -> *
```

```
c :: k
```

```
k :: *
```

```
f :: k -> *
```

```
b :: k
```

```
data Q :: forall (a :: f b) (c :: k) . f c -> *
```

```
k :: *
```

```
f :: k -> *
```

```
b :: k
```

data Q :: forall (a :: f b) (c :: k) . f c -> *

data Q :: forall (f::?) (b::?) (k::?) forall (a :: f b) (c :: k) . f c -> *

```
data Q :: forall (a :: f b) (c :: k) . f c -> *
```

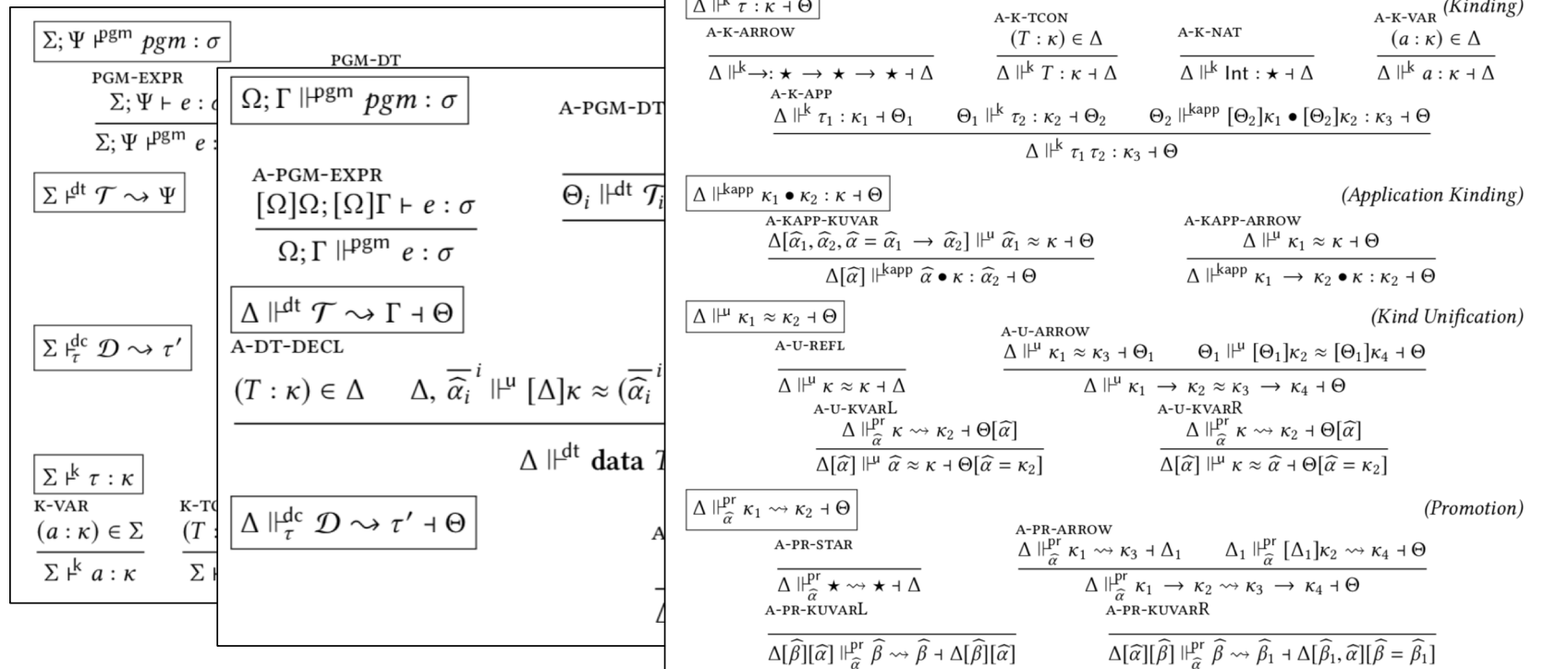
```
data Q :: forall (k::?) (f::?) (b::?) forall (a :: f b) (c :: k) . f c -> *
```

data Q :: forall (a :: f b) (c :: k) . f c -> *

data Q :: forall (k :: *) (f :: k -> *) (b :: k) forall (a :: f b) (c :: k) . f c -> *

Inferring Datatypes

- If you are a type theorist...



Inferring Datatypes

- If you are a GHC hacker/developer...
 - Further language extensions
 - How our work relates to GHC